University Course Management System

Theip Lual, Jeremy Roalef, Michael Roberts

Department of Computer Information Technology, SUNY Morrisville

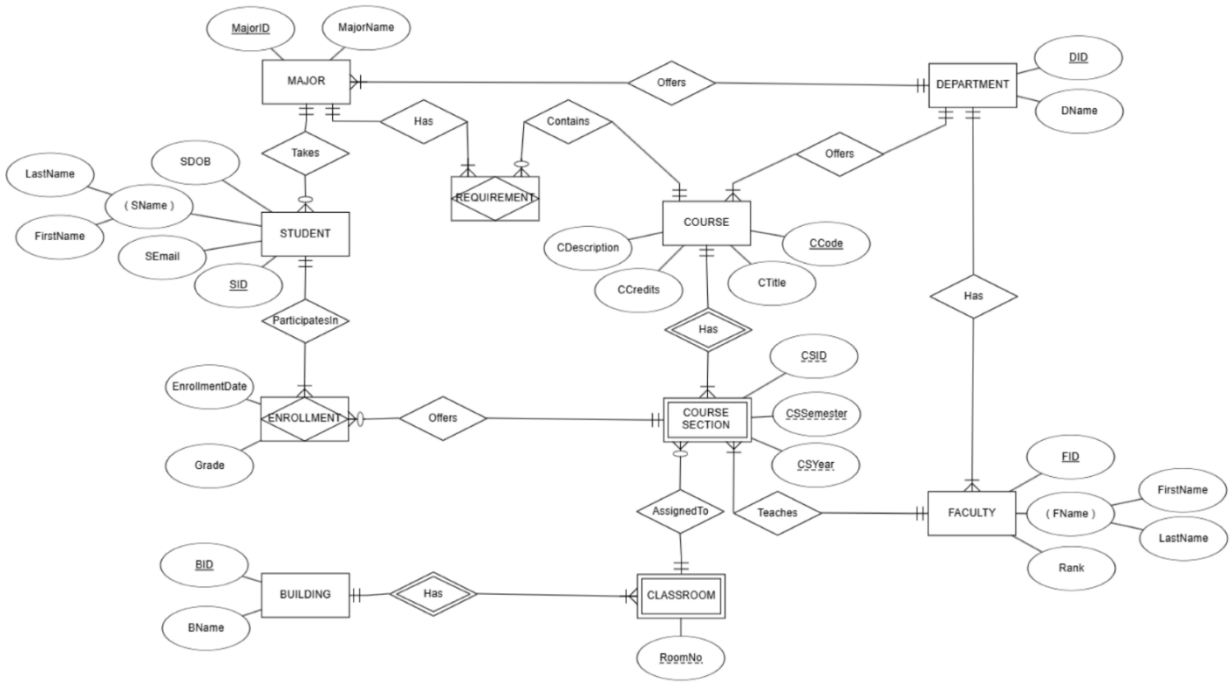
CITA 245: Introduction to Database Concepts

Mustafa A. Elfituri, Ph.D.

December 5th, 2024

Table of Contents

Entity-Relationship Diagram (ERD)

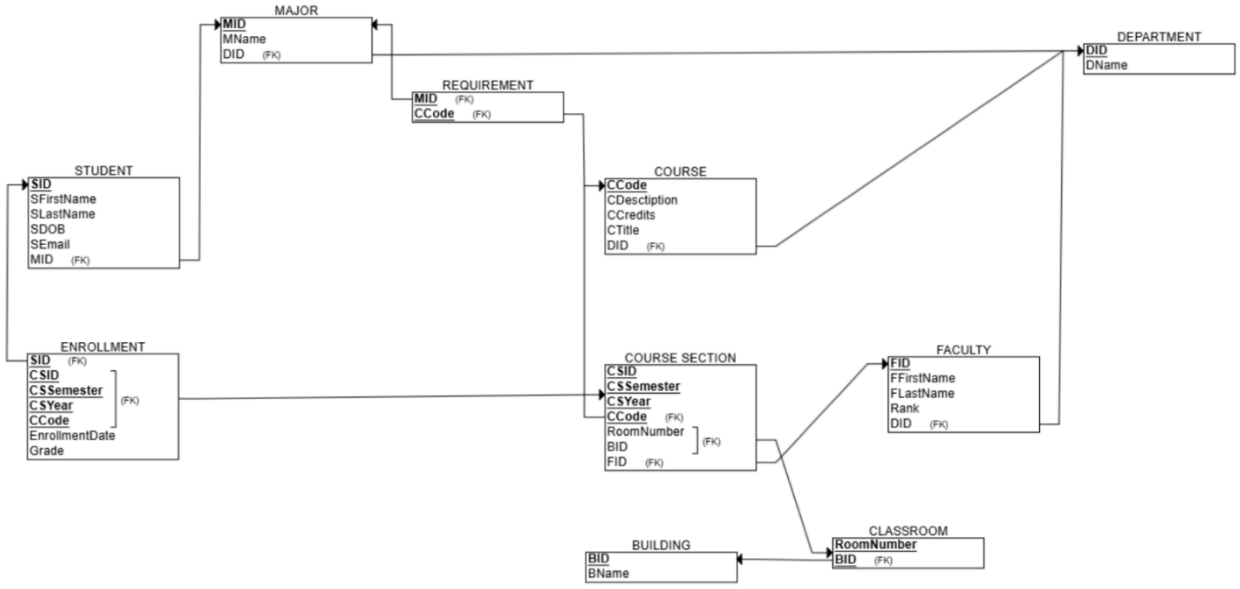


Notes

Creating an Entity-Relationship Diagram (ERD) for these requirements involved navigating complex relationships and ensuring their accurate representation. Many-to-many relationships, such as those between students and course sections or majors and courses, required the creation of associative entities like enrollment or major-course requirements. Additionally, hierarchical relationships, such as majors being linked to departments, demanded careful modeling to avoid redundancy while maintaining clarity. Ensuring adherence to normalization principles further complicated the process; for example, splitting attributes into separate entities like Building and Classroom improved normalization but added design complexity. Composite keys, used in entities like sections and classrooms, preserved data integrity but posed additional challenges during implementation.

Another significant challenge was interpreting and correctly implementing the business rules within the ERD. Rules such as "each course section is assigned to one classroom, but a classroom does not need a course section assigned" or "each major does not need to be taken by any students" required precise representation of optional relationships. Effectively modeling these rules, whether through NULL values or optional participation constraints, necessitated a careful balance between accuracy and usability, highlighting the importance of thoughtful design choices in database creation.

Relational Schema



Notes

Translating the ERD into a relational schema began with defining each entity as a table, ensuring the selection of primary keys guaranteed uniqueness and integrity. Composite keys were employed for complex relationships, such as in the Section table, which combines csid, cssemester, csyear, and ccode. Similarly, Enrollment leveraged composite keys to capture the

many-to-many relationship between Students and Course Sections. Adhering to normalization principles, attributes were separated into related tables to reduce redundancy. For example, splitting Building and Classroom allowed efficient room tracking without duplicating building details. Many-to-many relationships, like those between Courses and Majors, were handled with associative tables, such as Requirement, ensuring clarity and integrity.

Foreign key constraints were used extensively to enforce dependencies identified in the ERD. For instance, Classroom includes foreign keys referencing Building, ensuring valid associations. Hierarchical relationships, like those between Faculty, Major, and Department, were also captured through foreign keys. Challenges arose in implementing business rules, such as linking course sections to valid classrooms and faculty members, which required composite foreign keys in the Section table.

Data Dictionary

STUDENT

SID	SFirstName	SLastName	SEmail	SDOB	MID (FK)
1	Jimmy	Mas	minIXmas@IMT.co	4/4/2004	1
2	Stacey	Macey	aceyM4cey@IMT.c	1/2/2003	4
3	Manny	Smith	MSmith@IMT.com	12/12/1812	2
4	John	Dover	JoeDoe@IMT.com	11/11/2001	1
5	Steve	Block	itevenson@IMT.cor	10/8/2000	3

MAJOR

MID	MName	DID (FK)
1	Game Design	1
2	Automotive	5
3	Nursing	4
4	Cyber Security	1
5	CIT	1

Requirement

CCODE(FK)	MID(FK)
4	1
1	2
2	3
5	4
4	5

ENROLLMENT

SID (FK)	CSID (FK)	CSSemester (FK)	CSYear (FK)	CCode (FK)	EDate	EGrade
1	1	SPRING	2023	1	1/29/2023	70
1	1	FALL	2024	2	8/29/2023	65
3	1	SPRING	2025	3	1/29/2024	96
2	1	FALL	2023	4	8/29/2023	93
4	1	FALL	2023	2	1/29/2023	30
4	1	SPRING	2025	3	1/29/2023	86
5	1	FALL	2025	5	8/29/2023	67

COURSE

CCODE	CTitle	CDesc	CCredits	DID (FK)
1	MONEY101	Get Money	3	1
2	PSYCH 101	Get Mental	3	3
3	AUTO 300	Get Cars	3	5
4	CITA 650	Get Computers	4	4
5	CITA 750	et MORE Compute	4	2
6	BSAD 100	Be Sad	3	1
7	FYE 100	irst Year Experienc	1	3

DEPARTMENT

DID	DName
1	TECH
2	LIT
3	BIZ
4	GEN
5	AUTO 300

COURSE SECTION

CSID	CCode (FK)	CSSemester	CSYear	FID(FK)	RoomNo (FK)	BID (FK)
1	1	SPRING	2023	1	101	1
1	2	FALL	2024	2	102	5
1	3	SPRING	2025	4	102	5
1	4	FALL	2023	5	201	1
1	5	FALL	2025	4	101	2
1	6	SPRING	2026	3	502	4
1	7	SPRING	2026	3	202	3

FALCULTY

FID	FFirstName	FLastNme	Rank	DID (FK)
1	Daniel	Dobes	Full Professor	4
2	Dizzy	Gear	Full Professor	3
3	Doakes	Butch	Associate Professo	1
4	Dantes	Rank	Assistant Professo	2
5	Alexandres	Horse	Assistant Professo	5

CLASSROOM

RoomNo	BID (FK)
101	1
102	5
201	1
202	5
101	2
102	2
202	3
502	4

BUILDING

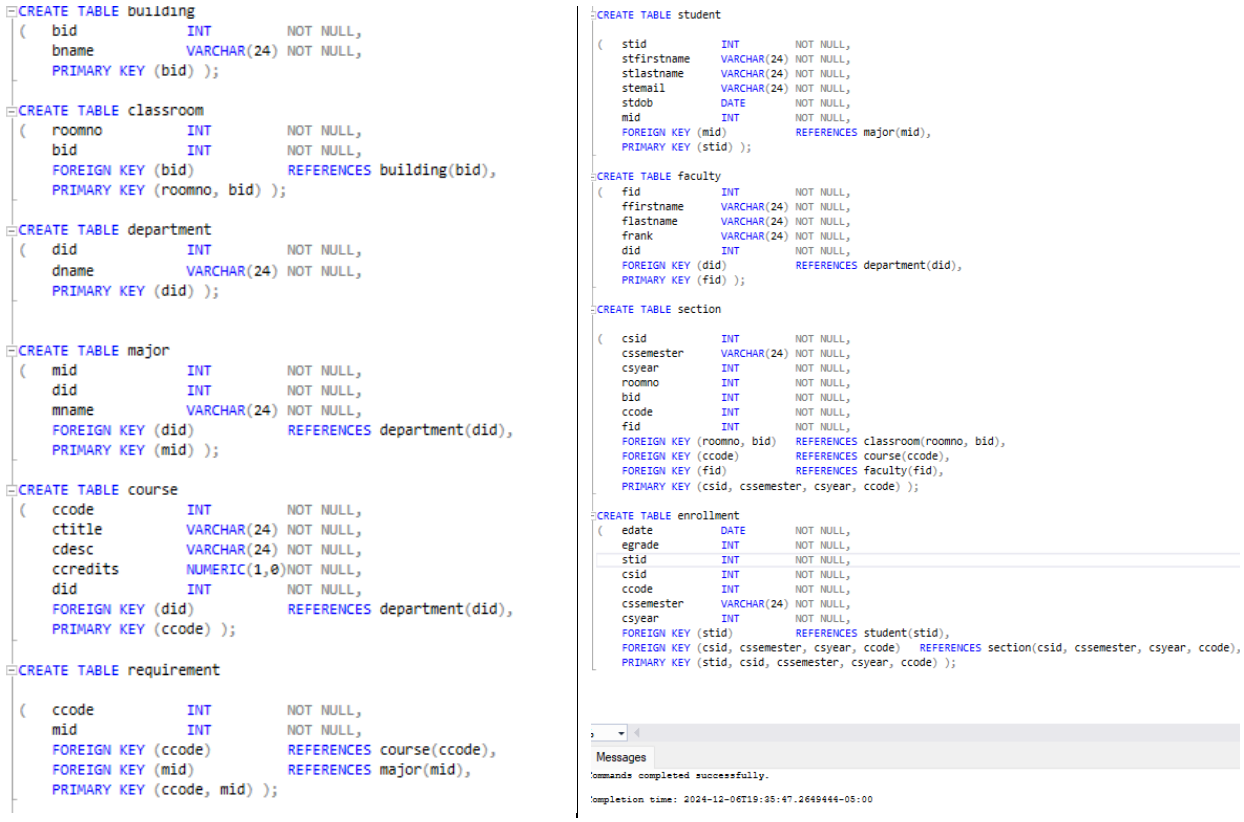
BID	BName
1	Dodge Hall
2	Barlton
3	Amigna Hall
4	Dome Hall
5	Morris Hall

Notes

To prepare for the upcoming queries, we intentionally populated each table with a substantial number of entries, ensuring the data was diverse and sufficiently distributed for effective querying. Additionally, we refined our column naming conventions, aligning them with a clear and consistent data dictionary that is reflected in the SQL database. This approach eliminated potential conflicts between tables, making it easier to distinguish between different ID types and naming conventions. For instance, it became clear whether we were referencing a student's name or a faculty member's name, avoiding the need to clarify the table or type in each query.

However, despite these efforts, minor spelling errors, such as "Falculty" instead of "Faculty," slipped through during the creation process. These mistakes were only discovered after the database was completed, necessitating a redo of the affected tables. Rather than retaining the error and perpetuating the issue in all future references to "Faculty," we opted to correct it and rebuild the table to ensure long-term consistency and accuracy.

SQL Create Table Statements



```
CREATE TABLE building
(   bid             INT         NOT NULL,
    bname           VARCHAR(24) NOT NULL,
    PRIMARY KEY (bid) );

CREATE TABLE classroom
(   roomno          INT         NOT NULL,
    bid             INT         NOT NULL,
    FOREIGN KEY (bid)           REFERENCES building(bid),
    PRIMARY KEY (roomno, bid) );

CREATE TABLE department
(   did             INT         NOT NULL,
    dname           VARCHAR(24) NOT NULL,
    PRIMARY KEY (did) );


CREATE TABLE major
(   mid             INT         NOT NULL,
    did             INT         NOT NULL,
    mname           VARCHAR(24) NOT NULL,
    FOREIGN KEY (did)           REFERENCES department(did),
    PRIMARY KEY (mid) );

CREATE TABLE course
(   ccode           INT         NOT NULL,
    ctitle          VARCHAR(24) NOT NULL,
    cdesc           VARCHAR(24) NOT NULL,
    ccredits        NUMERIC(1,0)NOT NULL,
    did             INT         NOT NULL,
    FOREIGN KEY (did)           REFERENCES department(did),
    PRIMARY KEY (ccode) );

CREATE TABLE requirement

(   ccode           INT         NOT NULL,
    mid             INT         NOT NULL,
    FOREIGN KEY (ccode)         REFERENCES course(ccode),
    FOREIGN KEY (mid)           REFERENCES major(mid),
    PRIMARY KEY (ccode, mid) );
```

```
CREATE TABLE student

(   stid            INT         NOT NULL,
    stfirstname     VARCHAR(24) NOT NULL,
    stlastname      VARCHAR(24) NOT NULL,
    stemail         VARCHAR(24) NOT NULL,
    stdob           DATE        NOT NULL,
    mid             INT         NOT NULL,
    FOREIGN KEY (mid)           REFERENCES major(mid),
    PRIMARY KEY (stid) );

CREATE TABLE faculty
(   fid             INT         NOT NULL,
    ffirstname      VARCHAR(24) NOT NULL,
    flastname       VARCHAR(24) NOT NULL,
    frank           VARCHAR(24) NOT NULL,
    did             INT         NOT NULL,
    FOREIGN KEY (did)           REFERENCES department(did),
    PRIMARY KEY (fid) );

CREATE TABLE section

(   csid            INT         NOT NULL,
    cssemester      VARCHAR(24) NOT NULL,
    csyear          INT         NOT NULL,
    roomno          INT         NOT NULL,
    bid             INT         NOT NULL,
    ccode           INT         NOT NULL,
    fid             INT         NOT NULL,
    FOREIGN KEY (roomno, bid)   REFERENCES classroom(roomno, bid),
    FOREIGN KEY (ccode)         REFERENCES course(ccode),
    FOREIGN KEY (fid)           REFERENCES faculty(fid),
    PRIMARY KEY (csid, cssemester, csyear, ccode) );

CREATE TABLE enrollment
(   edate           DATE        NOT NULL,
    egrade          INT         NOT NULL,
    stid            INT         NOT NULL,
    csid            INT         NOT NULL,
    ccode           INT         NOT NULL,
    cssemester      VARCHAR(24) NOT NULL,
    csyear          INT         NOT NULL,
    FOREIGN KEY (stid)          REFERENCES student(stid),
    FOREIGN KEY (csid, cssemester, csyear, ccode)   REFERENCES section(csid, cssemester, csyear, ccode),
    PRIMARY KEY (stid, csid, cssemester, csyear, ccode) );
```

```
Messages
Commands completed successfully.

Completion time: 2024-12-06T19:35:47.2649444-05:00
```

Notes

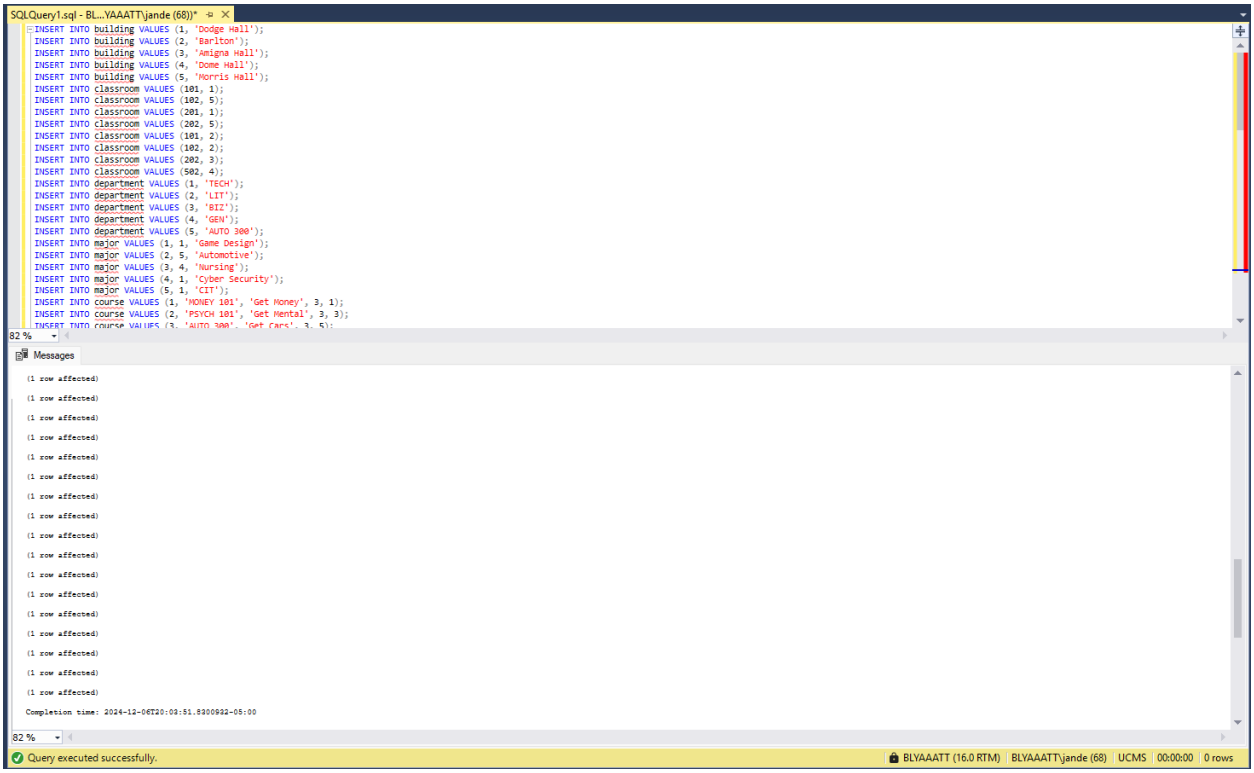
The process of translating the ERD and relational schema into SQL tables highlighted several areas of difficulty and required careful attention to detail. While the conceptual foundation provided by the ERD and schema was strong, practical issues arose during implementation, particularly regarding data types, constraints, and table design. One significant challenge involved selecting appropriate data types for various attributes. For example, initially representing “Enrollment grade” as a letter grade (e.g., A, B, C) caused issues during querying, particularly when trying to handle grades as an integer (e.g., 0–100). This discrepancy required multiple revisions to the table structure to make queries simpler and more logical for real-world grading systems. Similarly, using VARCHAR(24) for many string attributes, such as names and

email addresses, required balancing storage efficiency and practical constraints while ensuring the lengths were sufficient for all possible values.

Ensuring adherence to normalization principles was another key aspect of the process. This involved breaking data into multiple interrelated tables, such as Building, Classroom, Major, and Department, to eliminate redundancy. However, this separation introduced the need for complex foreign key relationships. Additionally, the use of composite keys in tables like Section and Enrollment added further complexity. For instance, the composite primary key in Section (csid, cssemester, csyear, ccode) ensures uniqueness across multiple dimensions but makes certain queries more cumbersome, especially when joining tables.

The relationships between tables required extensive use of foreign keys to maintain data integrity. For example, the Classroom table references Building through a composite key (roomno, bid), ensuring that each classroom is tied to a valid building. Similarly, the Enrollment table references Section using a composite key to ensure that enrollment records align with valid course sections. Mapping these constraints from the ERD to SQL was challenging, as errors in key definitions could lead to cascading data integrity issues.

SQL Insert Statements



Notes

The process of inserting sample data into the database revealed practical challenges in maintaining consistency and enforcing relationships within the relational schema. While tables like Building and Classroom were relatively straightforward, composite primary keys (e.g., roomno and bid) in the Classroom table added complexity, requiring extra care to ensure unique combinations. Similarly, dependencies in tables like Major and Course, which relied on foreign keys to departments, emphasized the importance of a precise insertion order to maintain referential integrity. The Requirement table, with its many-to-many relationships, highlighted the difficulties of managing such connections in larger datasets. Errors in formatting, such as inconsistent faculty ID types, further illustrated the need for standardized input formats to avoid schema conflicts.

The Section and Enrollment tables presented additional challenges due to their composite primary keys and multiple foreign key dependencies. For example, ensuring valid references for classrooms, faculty, and courses required careful validation. The decision to use letter grades for egrade in Enrollment was a meaningful refinement, aligning the schema more closely with real-world grading practices and simplifying queries. However, managing dependencies and maintaining data integrity across related tables proved resource-intensive, particularly for complex relationships. Overall, the data population phase underscored the importance of careful planning, data validation, and iterative schema refinements to ensure the database's accuracy and usability.

SQL Queries

Query 1: List all students and their respective majors

	firstName	lastName	majorName
1	Jimmy	Mas	Game Design
2	Stacey	Macey	Cyber Security
3	Manny	Smith	Automotive
4	John	Dover	Game Design
5	Steve	Block	Nursing

Query 2: Retrieve the schedule of all class sections, including course names, times, and assigned faculty.

	Course	Section	ProfessorFirstName	ProfessorLastName	Semester	Year
1	CITA 650	1	Alexandres	Horse	FALL	2023
2	PSYCH 101	1	Dizzy	Gear	FALL	2024
3	CITA 750	1	Dantes	Rank	FALL	2025
4	MONEY 101	1	Daniel	Dobes	SPRING	2023
5	AUTO 300	1	Dantes	Rank	SPRING	2025
6	BSAD 100	1	Doakes	Butch	SPRING	2026
7	FYE 100	1	Doakes	Butch	SPRING	2026

Query 3: Find all students enrolled in a specific class section

	FirstName	LastName	Course	Section
1	Jimmy	Mas	PSYCH 101	1
2	John	Dover	PSYCH 101	1

Query 4: Display the average grade for a specific class section

	ctitle	ClassAverageGrade
1	PSYCH 101	50

Query 5: List all faculty members teaching in a specific semester

	Semester	Year	FirstName	LastName
1	SPRING	2026	Doakes	Butch
2	SPRING	2026	Doakes	Butch

Query 6: Generate a report of all classrooms being used in a specific semester

	Semester	Year	Building	RoomNum
1	SPRING	2026	Dome Hall	502
2	SPRING	2026	Amigna Hall	202

Query 7: Retrieve all courses offered by a specific department

	Course	Department
1	PSYCH 101	BIZ
2	FYE 100	BIZ

Query 8: Identify students who are enrolled in more than three courses

	FirstName	LastName

Query 9: Show the total number of students enrolled in each department

	Department	StudentsInDepartment
1	TECH	3
2	LIT	0
3	BIZ	0
4	GEN	1
5	AUTO 300	1

Conclusion

Developing the University Course Management System was a comprehensive exercise in database design and implementation, providing critical insights into the challenges of creating a robust and scalable system. Beginning with the Entity-Relationship Diagram (ERD), the project required careful navigation of complex relationships, such as many-to-many connections

between students and course sections or hierarchical links between majors and departments. Normalization principles guided the design to eliminate redundancy, though this often added complexity, as seen in splitting attributes into separate entities like Building and Classroom. Translating the ERD into relational schemas and SQL tables revealed additional challenges, such as selecting appropriate data types, managing composite keys, and enforcing foreign key dependencies to maintain data integrity. Addressing nuanced business rules, such as optional participation constraints, required thoughtful modeling, ensuring the database was both accurate and practical for real-world scenarios.

The implementation phase further highlighted the importance of data validation, standardization, and iterative refinements. For example, maintaining consistency in naming conventions and composite keys proved vital to ensuring query accuracy and referential integrity. Practical challenges, such as shifting from numerical to letter-based grading systems, emphasized the need for adaptable design that aligns with user requirements. Additionally, creating sample data to populate the database exposed the complexities of enforcing relationships within the relational schema, particularly in tables like Section and Enrollment with their composite primary keys. This project underscored the significance of meticulous planning, normalization, and validation in crafting a functional database. It also provided invaluable experience in solving real-world challenges, preparing us to handle intricate database systems with confidence and precision in the future.